

MS&E 342 Final Project

Synthetic Data Generation Using Topological Loss for Deep Hedging

Björn Thor Stefánsson

Sondre Rogde

June 10, 2026

Abstract

We study synthetic financial time series generation for deep hedging. Building on the CoFinDiff transformer-based diffusion architecture [8], we add a topological regularization term to the training objective. The regularizer penalizes synthetic return sequences whose H_1 persistence landscape deviates from that of real training data, motivated by Gidea and Katz [5], who showed that persistence landscapes provide early warning signals of market crashes. We train three deep hedging agents: one on synthetic data from the base diffusion model, one on data from the topologically regularized model, and one on real data alone as a baseline. Agents are evaluated on held-out real market data via CVaR and drawdown. Our contribution is to test whether encoding topological structure into the data generation process translates into better hedging performance on real data, especially in stress scenarios.

1 Introduction and Related Work

Generating realistic synthetic financial data is a long-standing challenge. Asset returns exhibit well-documented empirical properties such as fat tails, volatility clustering, and the leverage effect [4]. Classical stochastic models such as Geometric Brownian Motion (GBM) and the Heston model offer analytical tractability but struggle to capture these properties. Deep generative models have emerged as a flexible alternative, with Variational Autoencoders, GANs, and more recently diffusion models all being applied to financial time series.

Diffusion models have achieved state-of-the-art quality in image synthesis [6] and have recently been adapted for financial data. The CoFinDiff model [8] introduces a transformer-based Denoising Diffusion Probabilistic Model (DDPM) for return sequences, with an optional conditioning mechanism that allows incorporating market views. We use the unconditional version and build directly on this architecture.

A good synthetic data model is ultimately judged by whether its output is useful for downstream tasks [9]. The task we care about is deep hedging [2], where a neural network is trained to hedge financial derivatives against a simulated market. Deep hedging models require large amounts of market data, often more than historical records can provide, making high-quality synthetic data valuable as it enables data augmentation and can expose the hedger to market regimes that are underrepresented in the historical record.

Beyond matching first and second moments, we want synthetic data that captures structural features of real markets. Gidea and Katz [5] show that persistent homology of sliding-window delay embeddings of return series predicts market crashes. In particular, the L^1 norm of the H_1 persistence landscape rises sharply in the 250 trading days before major market dislocations. Hedging is most critical during these stress periods, so it is natural to ask whether a synthetic data model that matches the topological structure of real data produces better-calibrated hedging agents.

We test this by augmenting the DDPM training loss with a differentiable topological regularization term. The regularizer is differentiable almost everywhere, using the subgradient approach of Hofer et al. [7] and Carrière et al. [3], and is computed via the GUDHI library. We compare hedging performance under three training regimes across 30 liquid S&P 500 stocks over a 19-year period.

2 Method

2.1 Denoising Diffusion Probabilistic Model

We follow the DDPM formulation of Ho et al. [6] as adapted in CoFinDiff [8]. The backbone is a six-layer transformer with model dimension 256 operating on sequences of log-returns $x_0 \in \mathbb{R}^T$, with $T = 252$ trading days.

The forward process adds Gaussian noise over $T_{\text{diff}} = 1000$ steps under a linear noise schedule:

$$q(x_t | x_0) = \mathcal{N}(\sqrt{\bar{\alpha}_t} x_0, (1 - \bar{\alpha}_t)I), \quad \bar{\alpha}_t = \prod_{s=1}^t (1 - \beta_s).$$

The reverse process is learned by predicting the noise ϵ added at each step:

$$\mathcal{L}_{\text{DDPM}} = \mathbb{E}_{x_0, \epsilon, t} [\|\epsilon - \epsilon_\theta(x_t, t)\|^2].$$

We use an open-source implementation of FinDiffusion.¹ We generate unconditional sequences only, i.e. we do not use the conditioning mechanism. We do this since our goal is to isolate the effect of the topological regularizer rather than study market-view conditioning.

2.2 Topological Regularization

Delay embedding. Given a log-return sequence $x = (x_1, \dots, x_T)$ and an embedding dimension d , the Takens delay embedding maps it to a point cloud in $\mathbb{R}^d$:

$$\mathcal{P}_d(x) = \{(x_t, x_{t+1}, \dots, x_{t+d-1}) : t = 1, \dots, T - d + 1\}.$$

For $T = 252$ and $d = 3$ (following Gidea and Katz [5]), this gives 250 points in $\mathbb{R}^3$. The map $x \mapsto \mathcal{P}_d(x)$ is linear, so gradients flow through it unchanged.

Vietoris-Rips persistent homology. We build the Vietoris-Rips filtration on $\mathcal{P}_d(x)$. At scale r , a simplex σ is included when all pairwise Euclidean distances among its vertices are at most r . As r grows from zero, topological features, such as connected components and loops, born and die. The H_1 persistence diagram records the birth and death of one-dimensional loops (1-cycles):

$$\text{Dgm}_1(x) = \{(b_i, d_i) : d_i > b_i, d_i < \infty\}.$$

We focus on H_1 because Gidea and Katz find these features to be the most predictive of market crises. The filtration range r_{max} is estimated as the 90th percentile of pairwise distances in a small sample of training sequences, which automatically adapts the scale to the data.

Persistence landscape. The persistence landscape [1] converts a diagram into a functional object that supports vector-space operations and gradient-based optimization. For each diagram point (b_i, d_i) define the tent function:

$$\Lambda_i(t) = \max(0, \min(t - b_i, d_i - t)).$$

The n -th landscape function is the n -th largest tent value at each t :

$$\lambda_n(t) = \text{kmax}_i \{\Lambda_i(t)\}, \quad n = 1, \dots, N_\lambda.$$

¹We use the implementation in <https://github.com/EmmanuelleB985/FinDiffusion>. The main structural difference is that this implementation omits the Haar Wavelet multi-scale decomposition used in the original paper.

Evaluated on a grid of $N_g = 50$ points over $[0, r_{\max}]$, this yields a landscape matrix $\lambda(x) \in \mathbb{R}^{N_\lambda \times N_g}$. Persistence landscapes live in a Banach space [1], so L^2 distances between them are well-defined and stable (Lipschitz continuous in the Bottleneck distance between diagrams).

Reference landscape. Before training begins, we compute the mean H_1 landscape over real training sequences (up to 500 are used in practice):

$$\bar{\lambda}^{\text{real}} = \frac{1}{|\mathcal{D}_{\text{train}}|} \sum_{x \in \mathcal{D}_{\text{train}}} \lambda(x) \in \mathbb{R}^{N_\lambda \times N_g}.$$

This is stored as a frozen buffer and is not updated during training.

Topological loss. During training, the predicted clean sequence is recovered from the noise prediction via:

$$\hat{x}_0 = \frac{x_t - \sqrt{1 - \bar{\alpha}_t} \epsilon_\theta(x_t, t)}{\sqrt{\bar{\alpha}_t}}.$$

The topological loss penalizes deviation of the generated landscape from the reference:

$$\mathcal{L}_{\text{topo}} = \frac{1}{B'} \sum_{b=1}^{B'} \|\lambda(\hat{x}_0^{(b)}) - \bar{\lambda}^{\text{real}}\|_F^2,$$

where $B' \leq B$ is a random sub-batch drawn at each step. The combined training objective is:

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{DDPM}} + \alpha_{\text{topo}} \cdot \mathcal{L}_{\text{topo}},$$

where α_{topo} is the weight given to the topological loss.

Differentiability. Computing the Vietoris-Rips complex involves sorting pairwise distances and running a boundary-matrix reduction algorithm. The output is which simplex creates or destroys a given loop and depends on this discrete ordering, so the map $x \mapsto \text{Dgm}_1(x)$ has no smooth gradient. The key insight of Hofer et al. [7] and Carrière et al. [3] is that the *value* at each critical simplex is differentiable almost everywhere, even though the *assignment* of critical simplices can jump. For an H_1 feature with birth edge (v_a, v_b) and death triangle (v_1, v_2, v_3) :

$$\begin{aligned} b_i &= \|p_{v_a} - p_{v_b}\|, & \frac{\partial b_i}{\partial p_{v_a}} &= \frac{p_{v_a} - p_{v_b}}{\|p_{v_a} - p_{v_b}\|}, \\ d_i &= \max_{(u,v) \in \sigma_i^d} \|p_u - p_v\|, & \frac{\partial d_i}{\partial p_u} &= \frac{p_u - p_v}{\|p_u - p_v\|} \text{ (longest edge only).} \end{aligned}$$

We identify the critical pairs in the forward pass using GUDHI, then treat them as fixed in the backward pass (standard subgradient / envelope theorem). The kmax in the landscape is implemented via `torch.topk`, which is differentiable. The full gradient chain is:

$$\hat{x}_0 \xrightarrow{\text{linear}} \mathcal{P}_3(\hat{x}_0) \xrightarrow{\text{Rips (custom grad)}} (b_i, d_i) \xrightarrow{\text{topk}} \lambda \xrightarrow{\text{MSE}} \mathcal{L}_{\text{topo}}.$$

Because computing the Rips complex on 250 points costs roughly $O(N^3)$, we apply the topological loss only every 5 training steps and use a sub-batch of size $B' = 16$ out of a full batch of 64. We also restrict the topological loss to timesteps $t \leq t_{\max}$ to avoid the numerical instability of $\hat{x}_0$ at high noise levels. When $\bar{\alpha}_t \approx 0$, dividing by $\sqrt{\bar{\alpha}_t}$ amplifies errors in the noise prediction and the recovered $\hat{x}_0$ is essentially pure noise, making the topological loss meaningless at those steps.

2.3 Stylized Facts

To sanity-check the generated data, we evaluate a set of well-known empirical properties of asset returns [4]:

- **Fat tails:** excess kurtosis and tail index.
- **Volatility clustering:** autocorrelation of squared returns.
- **Leverage effect:** negative correlation between returns and subsequent realized variance.
- **Absence of return autocorrelation:** near-zero linear autocorrelation in raw returns.

These are a sanity check, not the primary evaluation. Whether synthetic data is useful ultimately depends on downstream hedging performance [9]. We calculate these properties the same way they did in the CoFinDiff paper [8].

2.4 Deep Hedging Model

We follow the hedging setup of Buehler et al. [2] as described in CoFinDiff [8]. The hedging agent is a five-layer feedforward network with layer normalization and ReLU activations. At each time step it observes the current option price and outputs a hedge ratio for the underlying asset.

We hedge a European call option with a 30 trading-day horizon and a strike slightly out of the money (OTM). Liquid OTM options are a natural target because they carry significant gamma exposure relative to their premium and are the most common contracts that practitioners need to manage.

The agent is trained to minimize Conditional Value at Risk (CVaR) of the terminal profit-and-loss (P&L). CVaR, also known as expected shortfall, is the expected loss in the worst $\alpha\%$ of scenarios, which directly measures tail risk. CVaR is a natural risk metric for banks and institutional investors, especially after Basel III and MiFID II. We evaluate the trained agents on held-out real data (test set) using both CVaR at the 95% level and maximum drawdown of the cumulative P&L.

We compare three agents:

1. Trained on 10,000 synthetic sequences from the base DDPM.
2. Trained on 10,000 synthetic sequences from the DDPM + topological loss.
3. Trained on real historical data not overlapping with the test set.

Agent 3 is the benchmark. If the synthetic-data agents cannot match it, synthetic data generation adds no value in this setting. In practice one might want to use both synthetic and real data as there is no point in throwing away the real data when training the hedging agent. However, separating them isolates the contribution of each generation approach. The base DDPM agent (1 vs. 3) tests whether synthetic data is generally useful. The topo agent (2 vs. 1) tests whether topological regularization provides an additional benefit.

3 Data

We use daily closing prices for 30 liquid S&P 500 stocks spanning January 2005 to January 2024 (Table 5 in the Appendix lists all tickers). Log-returns are computed and normalized to zero mean and unit standard deviation per stock before training.

The 19-year span is split 60/20/20 into training, validation, and test sets. A 10% split would give roughly 300 observations per hold-out set (about one year of daily data), which is too short to draw reliable conclusions. Increasing to 20% gives approximately three to four years for validation and

testing, which includes the COVID-19 market crash in early 2020. This gives us a meaningful stress event for evaluating tail performance.

The data we use deviates from the original CoFinDiff paper [8], which uses 1-minute intraday data for 11 Japanese stocks over 7 years. Working at the daily scale over nearly two decades means the market regime changes more dramatically across the sample. Our hedging task is correspondingly specified at the daily scale using a 30 trading-day option, compared to the 300-minute horizon in the original paper. We do this because intraday data is expensive and hard to get. Also, the dataset in the original paper was much bigger and would require a lot more compute.

Sequences of length $T = 252$ (one trading year) are extracted with a sliding window of stride 21 (approximately one month). The stride limits the correlation between consecutive training samples and keeps computational cost manageable.

Table 1 reports summary statistics for the return series in each split.

Table 1: Summary statistics for daily log-returns across all 30 stocks. Excess kurtosis is the mean per-window excess kurtosis across all 252-day sliding-window sequences, consistent with Table 2.

Split	Mean (daily)	Std (daily)	Excess Kurtosis
Train (2005–2017)	0.000494	0.018866	4.91
Val (2017–2020)	0.000677	0.015639	6.03
Test (2020–2024)	0.000565	0.019941	4.80

4 Results

4.1 Generated Return Statistics

Table 2 compares distributional statistics of 10,000 generated sequences from each model against real training data. Mean and standard deviation are annualized by multiplying by 252 and $\sqrt{252}$, respectively.

Table 2: Distributional statistics of generated return sequences vs. real training data (annualized).

	Mean	Std	Excess Kurtosis
Real data	12.62%	30.31%	4.91
DDPM	14.87%	32.48%	7.59
DDPM + Topo	10.28%	33.97%	6.32

Both models land in a reasonable range. The base DDPM overshoots the real mean return a bit at 14.87% annualized versus 12.62% for real data, while the topo model undershoots slightly at 10.28%. Volatility is close across the board, with the real data at 30.31%, the DDPM at 32.48%, and the topo model at 33.97%. Neither gap is large enough to be a concern for the downstream hedging task. Both models also generate heavy-tailed distributions.

4.2 Stylized Facts

Table 3 reports stylized facts metrics. Figure 1 shows sample return paths for a visual sanity check. Both models pick up two of the four stylized facts. Fat tails are present in the generated sequences, which is the most important property for hedging since that is where the risk sits. Return

autocorrelation is also near zero in both cases, matching real returns. However, neither model reproduces volatility clustering or the leverage effect. These are more subtle temporal dependencies that require the model to learn how variance evolves across time. We did run some experiments where tweaking the loss pushed the model toward picking up these properties, but since the goal here is hedging performance rather than stylized fact matching, we did not pursue that direction further.

Table 3: Stylized facts for real and generated data, averaged across all 30 stocks. Volatility clustering is the lag-1 autocorrelation of squared returns. Leverage effect is the correlation between returns and subsequent squared returns. Return autocorr. is the lag-1 autocorrelation of raw returns (should be near zero).

	Kurtosis	Vol. clustering	Leverage effect	Return autocorr.
Real data	Yes	Yes	Yes	Yes
DDPM	Yes	No	No	Yes
DDPM + Topo	Yes	No	No	Yes

4.3 Hedging Performance

Table 4 reports hedging performance on held-out real test data. Figure 2 shows the terminal P&L distributions.

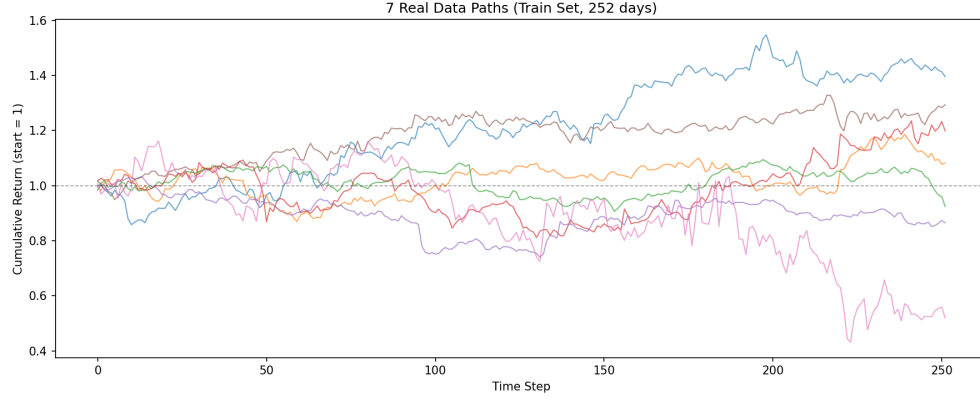
Both synthetic agents outperform the real-data agent on both metrics. The base DDPM achieves the best CVaR₉₅ at 7.88, compared to 8.20 for the topo agent and 8.81 for the real-data agent. The topo agent achieves the lowest max drawdown at 28.41, beating the base DDPM at 30.21 and the real-data agent at 32.96. Mean P&L is small and negative across all three agents, which is expected from a hedging strategy that pays a small premium to reduce tail risk.

The two synthetic models each lead on one metric. The base DDPM is better at CVaR₉₅, while the topo model is better at max drawdown. Max drawdown captures the worst single cumulative loss trajectory, which is arguably the most direct measure of crash-period performance. The fact that the topo model wins on that metric is consistent with the findings of [5].

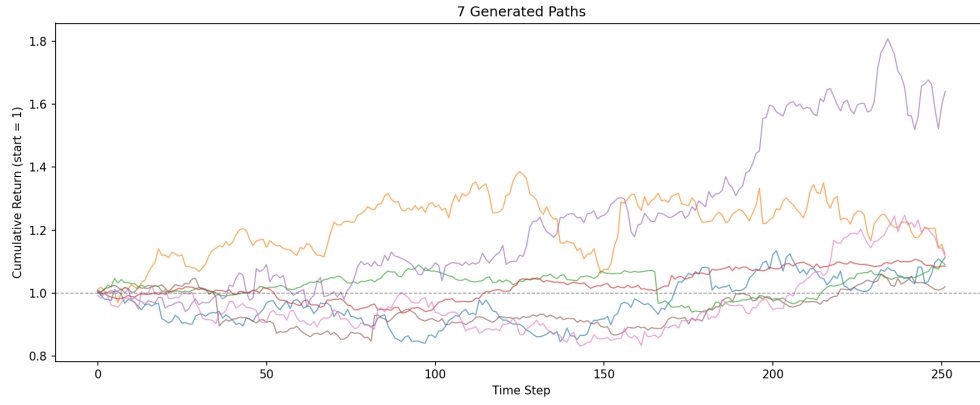
Table 4: Hedging performance on the real test set for European call options (30-day TTL, slightly OTM). CVaR₉₅ is the mean loss in the worst 5% of episodes. Lower is better for both metrics.

Training data	CVaR ₉₅	Max drawdown	Mean P&L
Real data	8.81	32.96	−1.55
Synthetic (DDPM)	7.88	30.21	−1.78
Synthetic (DDPM + Topo)	8.20	28.41	−1.59

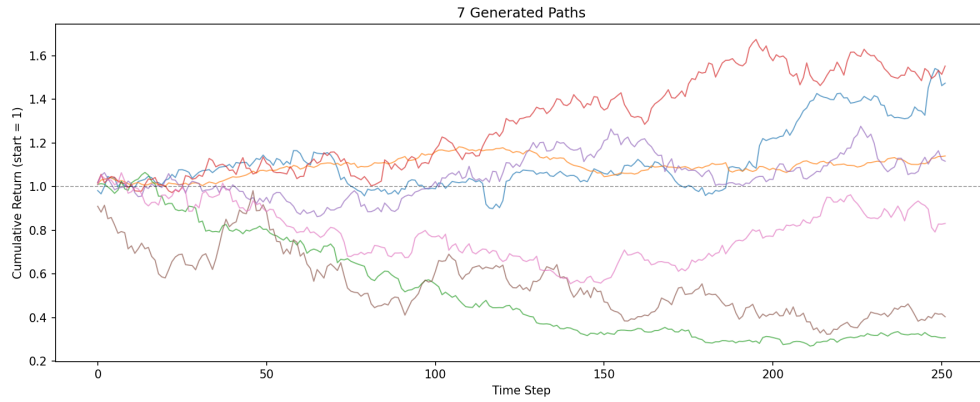
Figure 2 shows the terminal P&L distributions for all three agents. The two synthetic agents produce similar, tightly concentrated distributions centered near zero. The real-data agent, by contrast, has noticeably fatter tails in both directions: it occasionally achieves gains as large as 10% but suffers correspondingly larger losses in the left tail. Large profits are not a sign of a good hedging policy, as they suggest the agent took speculative directional positions rather than purely hedging. A large gain simply means the agent took significant unhedged directional exposure that happened to pay off in this evaluation sample. That same exposure is equally likely to produce a large loss on a different market path, which is precisely what the higher CVaR of the real-data agent captures. The wide tails most likely reflect underfitting as the real-data agent had much less



(a) Real data (train set)



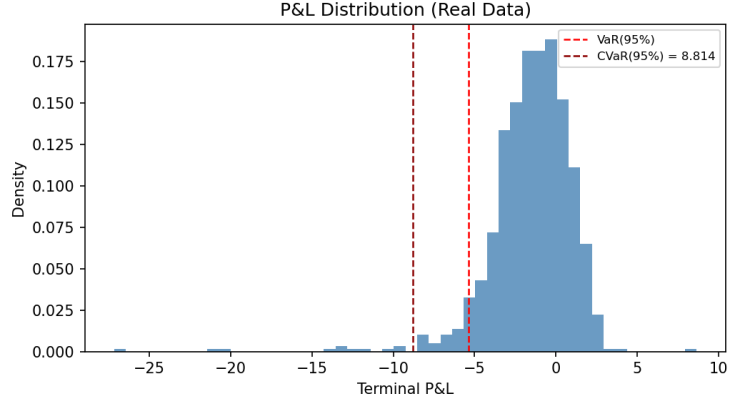
(b) Base DDPM



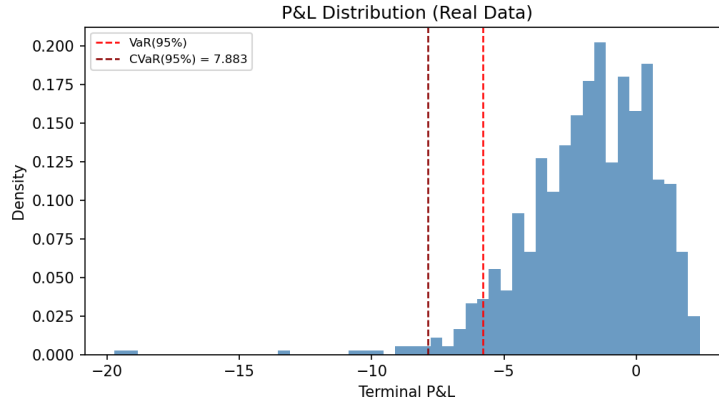
(c) DDPM with topological regularization

Figure 1: Sample 252-day cumulative return paths. Seven representative paths are shown per panel.

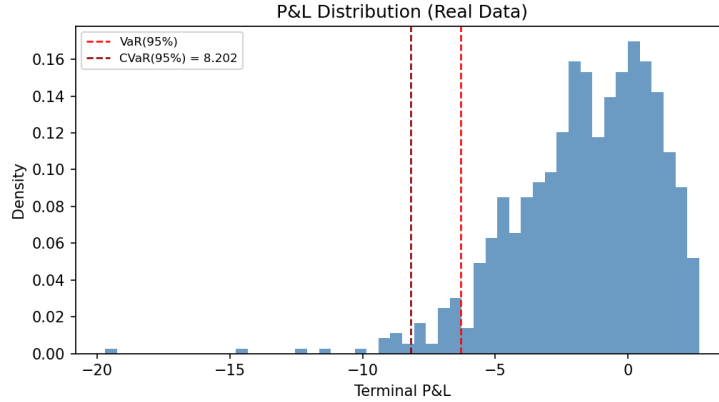
training data than the synthetic agents giving them far better coverage of the option's payoff profile. The ability to train on arbitrarily large data sets is a benefit of the synthetic agents.



(a) Real data



(b) Synthetic (DDPM)



(c) Synthetic (DDPM + Topological regularization)

Figure 2: Terminal P&L distributions for the three hedging agents evaluated on real test data. Left tail performance (CVaR) is the primary comparison axis.

5 Discussion and Future Work

Does synthetic data help? Yes. Both synthetic agents beat the real-data agent on both CVaR and max drawdown. The most likely explanation is that training on 10,000 synthetic sequences

gives the hedger much broader coverage of market regimes than the few hundred real sequences available in the training split. The COVID-19 crash falls in the test set and is exactly the kind of scenario that is rare in the historical record but common in synthetic data. Both synthetic agents handle this stress period better, which demonstrates that data generation via diffusion models is useful here.

Does topological regularization matter? The answer is mixed. The topo model does not beat the base DDPM on CVaR at the 95% level, where the base DDPM is actually better (7.88 vs. 8.20). However, the topo model achieves a notably lower max drawdown (28.41 vs. 30.21 for the base DDPM). Max drawdown captures the worst single cumulative loss trajectory in the test set, which is a direct measure of how the model performs during a market dislocation. This is consistent with the finding in Gidea and Katz [5] that persistence landscapes are informative right before major market crashes. The topological regularization appears to make the hedger more cautious in regimes that resemble crash precursors.

It is also worth noting that CVaR at 95% is a relatively mild tail metric. Many banks operate at 97.5% or 99%, and the topo model may have an advantage at those stricter thresholds. Checking whether the ranking flips at higher confidence levels would be a natural next step.

Limitations. The model operates on univariate return sequences, one stock at a time. In reality, hedging a portfolio requires capturing cross-asset dynamics. The delay-embedding point cloud extends naturally to the multivariate case by concatenating returns from multiple assets, which would let the H_1 features capture cross-correlation structure. Extending the diffusion model itself to multivariate generation is less straightforward, and we limited scope to the univariate case in order to match the original CoFinDiff setup and keep the comparison clean.

We also work at the daily scale, which means some microstructure effects studied in the original CoFinDiff paper are absent. Repeating the analysis at finer time scales could reveal richer topological structure and may be a more natural setting for the TDA approach.

Modified topological loss. One important methodological decision was how to penalize deviations from the reference landscape. Our main formulation (Section 2.2) uses a bidirectional MSE, which penalizes generated sequences both for having too little and too much topological structure relative to the training-data mean. One could argue that penalizing high-topology sequences is counterproductive for the hedging goal as sequences with *elevated* persistence are precisely the crash-precursor patterns that motivated our approach in the first place, and pulling the model away from them may reduce its coverage of stress scenarios. We therefore also ran experiments with a modified one-sided loss that only penalizes sequences whose landscape falls *below* the reference mean, leaving high-topology sequences unconstrained. The hedging results were qualitatively similar to those reported in Table 4, which suggests that the bidirectional penalty does not materially suppress crash-like sequences in practice. A detailed description of the modified loss is given in Appendix C.

Future work. Three directions stand out. First, extending to multivariate generation and evaluating portfolio-level hedging would be the most practically relevant next step. Second, a systematic study of how hedging performance varies with option moneyness and time to expiry would reveal where synthetic data is most and least useful. Third, the topological loss weight α_{topo} and the filtration dimension d are currently fixed at default values. A more thorough hyperparameter search, or adaptive weighting during training, could improve results.

References

- [1] Peter Bubenik. Statistical topological data analysis using persistence landscapes. *Journal of Machine Learning Research*, 16(3):77–102, 2015.
- [2] H. Buehler, L. Gonon, J. Teichmann, and B. Wood. Deep hedging. *Quantitative Finance*, 19(8):1271–1291, 2019.
- [3] Mathieu Carrière, Frédéric Chazal, Marc Glisse, Théo Lacombe, Martin Royer, and Yuhei Umeda. Optimizing persistent homology based functions. In *Proceedings of the 38th International Conference on Machine Learning, ICML 2021*, 2021.
- [4] R. Cont. Empirical properties of asset returns: stylized facts and statistical issues. *Quantitative Finance*, 1(2):223–236, 2001.
- [5] Marian Gidea and Yuri Katz. Topological data analysis of financial time series: Landscapes of crashes. *Physica A: Statistical Mechanics and its Applications*, 491:820–834, 2018.
- [6] Jonathan Ho, Ajay Jain, and Pieter Abbeel. Denoising diffusion probabilistic models. In *Proceedings of the 34th International Conference on Neural Information Processing Systems, NIPS ’20*, Red Hook, NY, USA, 2020. Curran Associates Inc.
- [7] Christoph Hofer, Roland Kwitt, Marc Niethammer, and Andreas Uhl. Deep learning with topological signatures. In *Advances in Neural Information Processing Systems*, volume 30, 2017.
- [8] Yuki Tanaka, Ryuji Hashimoto, Takehiro Takayanagi, Zhe Piao, Yuri Murayama, and Kiyoshi Izumi. Cofindiff: Controllable financial diffusion model for time series generation, 2025.
- [9] Lucas Theis, Aäron van den Oord, and Matthias Bethge. A note on the evaluation of generative models, 2016.

A List of Stocks

Table 5: The 30 S&P 500 stocks used in this study.

Ticker	Company	Ticker	Company
AAPL	Apple	NVDA	NVIDIA
MSFT	Microsoft	TSLA	Tesla
GOOGL	Alphabet	JPM	JPMorgan Chase
AMZN	Amazon	V	Visa
META	Meta	JNJ	Johnson & Johnson
WMT	Walmart	PG	Procter & Gamble
MA	Mastercard	HD	Home Depot
DIS	Disney	PYPL	PayPal
ADBE	Adobe	NFLX	Netflix
INTC	Intel	CSCO	Cisco
PFE	Pfizer	KO	Coca-Cola
PEP	PepsiCo	MRK	Merck
ABT	Abbott	TMO	Thermo Fisher Scientific
COST	Costco	AVGO	Broadcom
NKE	Nike	MCD	McDonald's

B Hyperparameters

Table 6: Key hyperparameters for the diffusion model and topological loss.

Parameter	Value	Notes
<i>Diffusion model</i>		
T_{diff}	1000	Diffusion timesteps
$\beta_{\text{start}}, \beta_{\text{end}}$	$10^{-4}, 0.02$	Linear noise schedule
d_{model}	256	Transformer model dimension
Layers	6	Transformer depth
Heads	8	Attention heads
Batch size	64	
Learning rate	10^{-4}	AdamW, cosine schedule, 5-epoch warmup
Epochs	30	
<i>Topological loss</i>		
d	3	Delay embedding dimension
N_{λ}	3	Number of landscape functions
N_g	50	Landscape grid resolution
α_{topo}	0.1	Topological loss weight
Apply every	5 steps	Frequency of persistence computation
Sub-batch B'	16	Samples per topo step
Ref. samples	500	Sequences used to compute $\bar{\lambda}^{\text{real}}$

C Modified Topological Loss

The main topological loss (Section 2.2) minimizes the squared Frobenius distance between each generated landscape and the reference mean $\bar{\lambda}^{\text{real}}$:

$$\mathcal{L}_{\text{topo}} = \frac{1}{B'} \sum_{b=1}^{B'} \|\lambda(\hat{x}_0^{(b)}) - \bar{\lambda}^{\text{real}}\|_F^2.$$

This is bidirectional as it penalizes a generated sequence equally whether its landscape is above or below the reference. A sequence with unusually high persistence, i.e. the signature of a crash precursor in Gidea and Katz [5], is penalized just as much as one with low persistence.

One-sided formulation. To avoid suppressing high-topology sequences, we defined a modified loss that only penalizes landscapes falling *below* the reference mean, element-wise:

$$\mathcal{L}_{\text{topo}}^+ = \frac{1}{B'} \sum_{b=1}^{B'} \|\max(0, \bar{\lambda}^{\text{real}} - \lambda(\hat{x}_0^{(b)}))\|_F^2,$$

where $\max(0, \cdot)$ is applied element-wise. Under $\mathcal{L}_{\text{topo}}^+$, the model is free to generate sequences with persistence *above* the reference without any gradient penalty, while still being pushed to include at least as much topological structure as the average real sequence. This is analogous to a lower-bound constraint on the persistence landscape rather than a target-matching constraint.

Implementation. The modification requires only replacing the residual $(\lambda - \bar{\lambda}^{\text{real}})$ with $\max(0, \bar{\lambda}^{\text{real}} - \lambda)$ before squaring. All other components of the gradient chain remain identical.

Variance term. In addition to penalizing the batch-mean landscape, our implementation adds a term that penalizes low topological *diversity*. Let $\hat{\sigma}$ denote the element-wise standard deviation of the batch landscapes $\{\lambda(\hat{x}_0^{(b)})\}$ and let $\bar{\sigma}^{\text{real}}$ be the corresponding standard deviation over the reference sequences. The variance term is also one-sided, it only fires when the batch spread falls below the reference:

$$\mathcal{L}_{\text{std}} = \|\max(0, \bar{\sigma}^{\text{real}} - \hat{\sigma})\|_F^2.$$

This penalizes topological mode collapse which is the scenario where the model generates sequences that are all topologically similar to one another. The combined loss used in practice is $\mathcal{L}_{\text{topo}} + \alpha_{\text{std}} \mathcal{L}_{\text{std}}$, with $\alpha_{\text{std}} = 0.5$ (Table 6).

Results. We trained the DDPM with $\mathcal{L}_{\text{topo}}^+$ using the same hyperparameters as the main model (Table 6). The resulting hedging performance was within the noise of the bidirectional model on both CVaR₉₅ and max drawdown. We therefore report the bidirectional version as the primary result, as it is simpler and symmetric, and reserve the one-sided variant as a potential refinement.